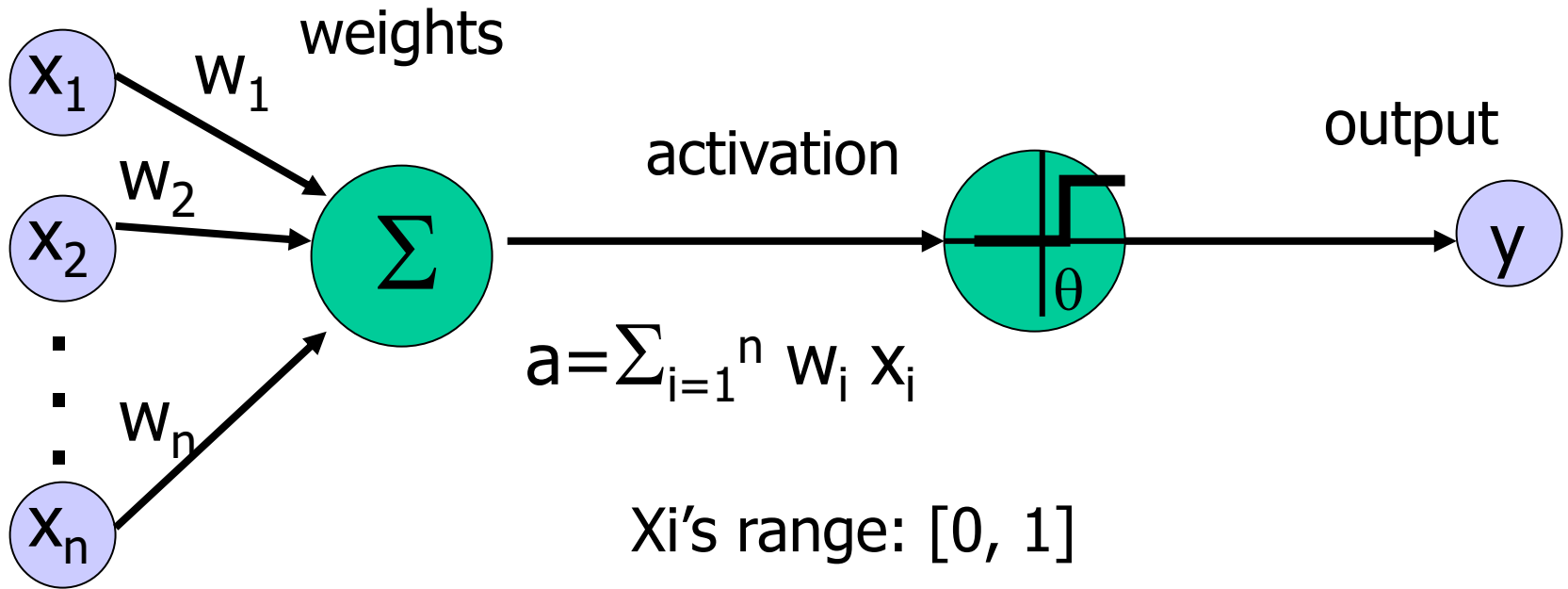

Artificial Neural Networks

Perceptron

inputs



$$y = \begin{cases} 1 & \text{if } a \geq \theta \\ 0 & \text{if } a < \theta \end{cases}$$

Perceptron (Example)

```
>>> from sklearn.linear_model import Perceptron
>>> per_clf = Perceptron(max_iter=1000, tol=1e3)
>>> per_clf.fit(X, y)
>>> print(per_clf.predict([[2, 0.5]]))
```

```
array([1])
```

In this example we initialize a perceptron called `per_clf` and call the `fit` method on some training data `X` of shape `(20, 2)` and their labels `y` of shape `(20, )`. We then ask our trained perceptron to predict the label for the input `[2, 0.5]`. Our input is classified as 1.

See: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Perceptron.html

Perceptron

```
class sklearn.linear_model.Perceptron(*, penalty=None, alpha=0.0001,  
                                       l1_ratio=0.15, fit_intercept=True, max_iter=1000, tol=0.001,  
                                       shuffle=True, verbose=0, eta0=1.0, n_jobs=None,  
                                       random_state=0, early_stopping=False,  
                                       validation_fraction=0.1, n_iter_no_change=5,  
                                       class_weight=None, warm_start=False)
```

- *max_iter* : int value that represents the number of training epochs in the *fit* method
- *tol* : float variable containing the stopping criterion. The iterations will stop when $(\text{loss} > \text{previous_loss} - \text{tol})$
- *penalty* : string that contains the regularization term to be used.
 - The possible values are `{'l1', 'l2', 'elasticnet', 'None'}`
- *alpha* : float value that multiplies the regularization term if regularization is used

Perceptron

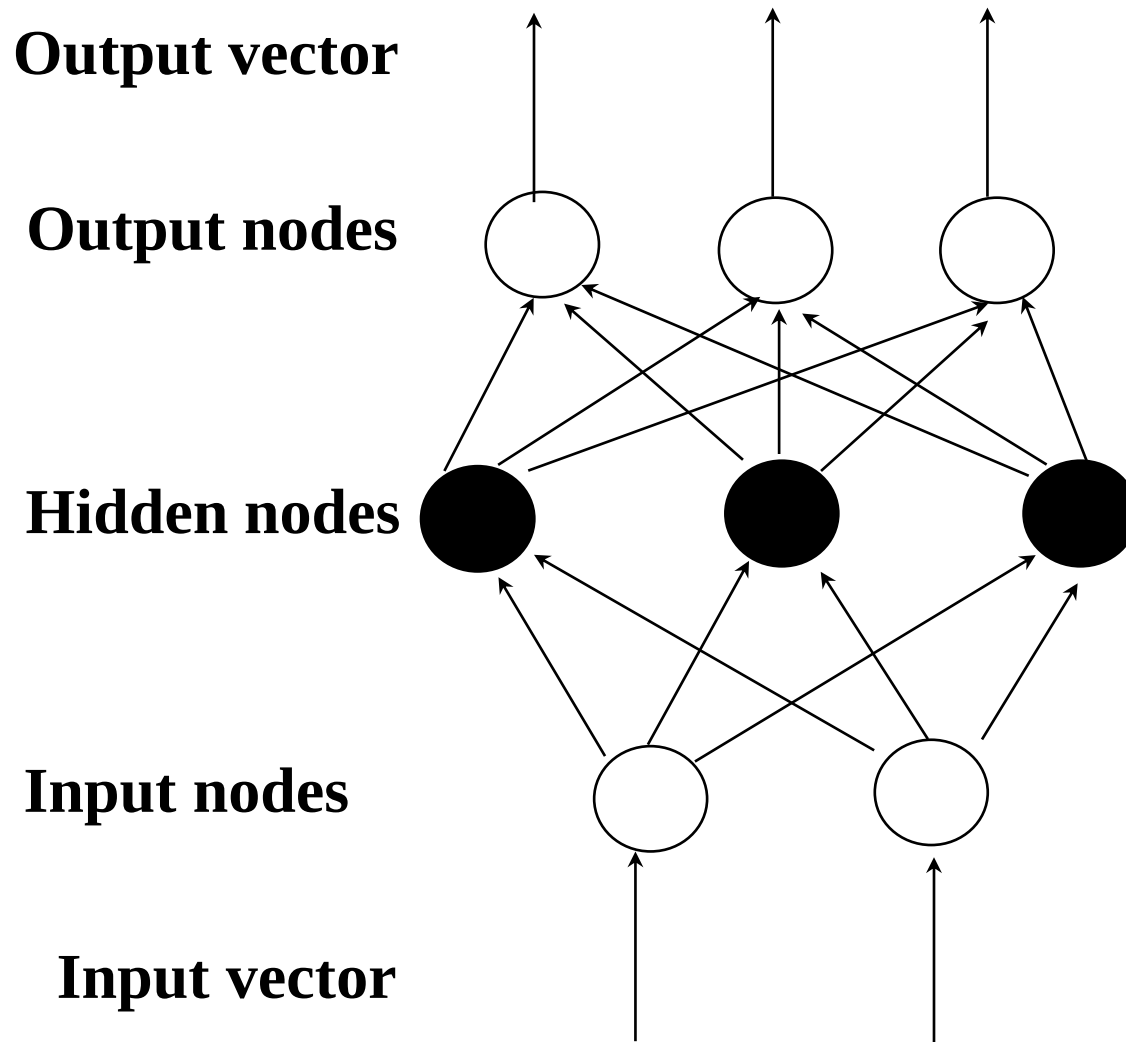
method **fit**(X, y, coef_init=None, intercept_init=None, sample_weight=None)

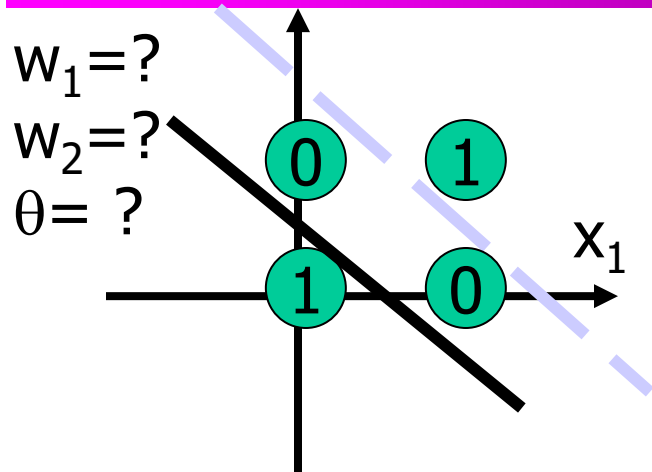
- X : the training data. This value can be an array or a sparse matrix of shape (n_samples, n_features).
- y : the target values. This value is a ndarray of shape (n_samples,)
- sample_weights : weights applied to individual samples. This value can be a list or an array of shape (n_samples,)

method **predict**(X1)

- X1 : Samples. This value can be an array or a sparse matrix of shape (n_samples, n_features).
- **returns** an array of shape [n_samples] that contains the predicted label per sample

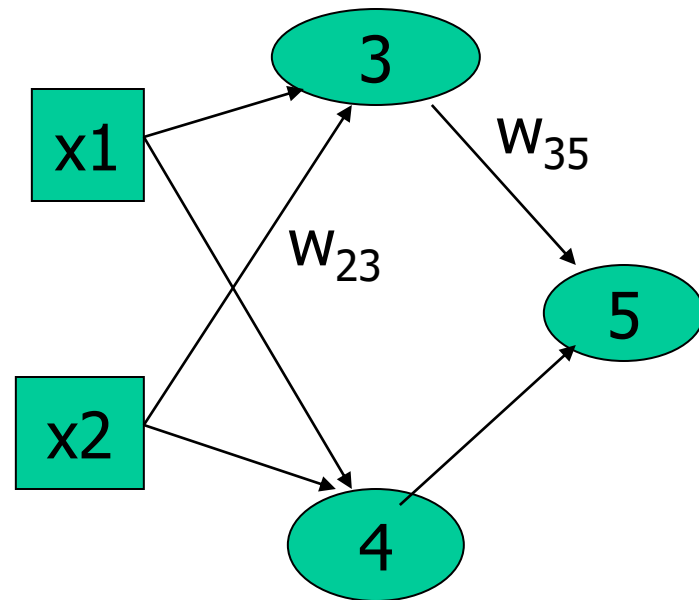
MLP





x_1	x_2	y	
0	0	0	
0	1	1	
1	0	1	
1	1	0	

- How to set the weights?



MLPClassifier

```
from sklearn.neural_network import MLPClassifier  
model = MLPClassifier(  
    activation='relu', max_iter=10000, hidden_layer_sizes=(4,2))  
model.fit(x_xor, y_xor)
```

Here we consider the MLP classifier in order to solve the XOR problem.

See: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

MLPClassifier

```
class sklearn.neural_network.MLPClassifier(hidden_layer_sizes=100,
      activation='relu', *, solver='adam', alpha=0.0001, batch_size='auto',
      learning_rate='constant', learning_rate_init=0.001, power_t=0.5,
      max_iter=200, shuffle=True, random_state=None, tol=0.0001,
      verbose=False, warm_start=False, momentum=0.9,
      nesterovs_momentum=True, early_stopping=False,
      validation_fraction=0.1, beta_1=0.9, beta_2=0.999, epsilon=1e-08,
      n_iter_no_change=10, max_fun=15000)
```

- hidden_layer_sizes: tuple, length = n_layers - 2, default=(100,)
- activation: {'identity', 'logistic', 'tanh', 'relu'}, default='relu'
- solver: {'lbfgs', 'sgd', 'adam'}, default='adam'
- batch_size: int, default='auto'
- max_iter: int, default=200

KERAS

Keras is an API designed for human beings, not machines. Keras follows best practices for reducing cognitive load: it offers consistent & simple APIs, it minimizes the number of user actions required for common use cases, and it provides clear & actionable error messages. It also has extensive documentation and developer guides.

Keras Sequential Model

A *Sequential* model is appropriate for a **plain stack of layers** where each layer has **exactly one input tensor and one output tensor**.

The layer class *Dense* implements the operation:

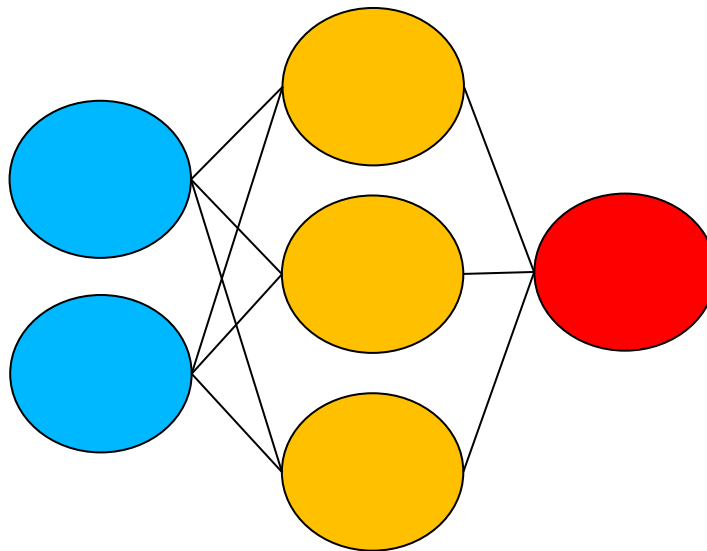
```
output = activation(dot(input, kernel) + bias)
```

where `activation` is the element-wise activation function passed as the `activation` argument, `kernel` is a weights matrix, and `bias` is a bias vector.

Sequential (Example)

```
>>> import tensorflow as tf
>>> from tensorflow import keras
>>> model = keras.models.Sequential()
>>> model.add(keras.layers.Dense(3, activation="relu"),
input_shape = (2,))
>>> model.add(keras.layers.Dense(1, activation="softmax"))
```

This is the network we are building in the example



Dense

```
class tf.keras.layers.Dense( units, activation=None, use_bias=True,  
kernel_initializer="glorot_uniform", bias_initializer="zeros",  
kernel_regularizer=None, bias_regularizer=None, activity_regularizer=None,  
kernel_constraint=None, bias_constraint=None)
```

- units : dimensionality of the output space. It's a positive integer.
- activation : activation function to use
 - 'linear' : no activation function
 - 'relu' : rectified linear activation function
 - 'sigmoid' : sigmoid activation function
- use_bias : boolean, wheather the layer uses the bias vector

Sequential

class tf.keras.**Sequential**(layers=None, name=None)

- layers : list of layers, usually Dense

method add(layer)

- Adds a layer instance on top of the layer stack

method fit(x=None, y=None, batch_size=None, epochs=1, verbose=1, callbacks=None, validation_split=0.0, validation_data=None, shuffle=True, class_weight=None, sample_weight=None, initial_epoch=0, steps_per_epoch=None, validation_steps=None, validation_batch_size=None, validation_freq=1, max_queue_size=10, workers=1, use_multiprocessing=False,)

- X: input data. It can be an array or a tensor.
- y : target data. It can be an array or a tensor. It must be consistent with X
- batch_size : number of samples per gradient update. It's an integer or None.
- epochs : number of epochs to train the model

Sequential

```
Model.evaluate( x=None, y=None, batch_size=None, verbose=1,  
sample_weight=None, steps=None, callbacks=None, max_queue_size=10,  
workers=1, use_multiprocessing=False, return_dict=False, )
```

- `x` : input samples. It can be an array or a tensor
- `y` : target data. It can be a ndarray or a tensor.
- `batch_size` : number of sample per batch. If not specified this value will default to 32
- **returns** scalar test loss (if the model has a single output and no metrics) or list of scalars (if the model has multiple outputs and/or metrics).

method summary()

- Prints a string summary of the network

Sequential (Example)

```
>>> model.summary()
```

```
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	
Param #		
=====		
dense (Dense)	(None, 3)	9
dense_1 (Dense)	(None, 1)	4
=====		

```
Total params: 13
```

```
Trainable params: 13
```

```
Non-trainable params: 0
```


Sequential (Example)

```
>>> history = model.fit(X_train, y_train, epochs=2,  
                        validation_data=(X_valid, y_valid))  
  
>>> model.evaluate(X_test, y_test)
```

```
Train on 55000 samples, validate on 5000 samples  
Epoch 1/2 55000/55000 [=====] - 2s 44us/sample -  
loss: 0.7226 - accuracy: 0.7641 - val_loss: 0.5073 - val_accuracy: 0.8320  
Epoch 2/2 55000/55000 [=====] - 2s 39us/sample -  
loss: 0.4844 - accuracy: 0.8321 - val_loss: 0.4541 - val_accuracy: 0.8478  
  
10000/10000 [=====] - 0s 21us/sample - loss: 0.3378  
- accuracy: 0.8781  
[0.33780701770782473, 0.8781]
```

In this example we are training our model using `X_train` and `y_train` data over 2 epochs adding the `accuracy` as a metric. We are also using `X_valid` and `y_valid` to validate our epochs, as we can see in the output report.

The `evaluate` method returns both the `loss` and the `accuracy`.